SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

Dry Run Evaluation

Code of Conduct:

I _P .Lakshmi Gayathri Navaneetha_ (192321169) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

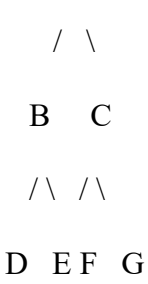
P.Gayathri
Signature of the student:

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
1	B, C	A
2	C, D, E	B
3	D, E, F, G	C
4	E, F, G	D
5	F, G	E
6	G	F
7	-	G

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

Answer:

BFS visits the nodes level by level, taking the node at the front of the queue each time and adding its children (left to right) at the back.

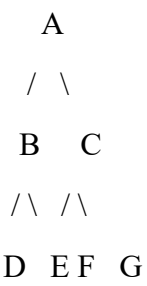
- **BFS traversal order:** A → B → C → D → E → F → G

Queue contents after each iteration (front on the left):

Iteration	Node taken from queue	Queue after the iteration
1	A	B, C
2	B	C, D, E
3	C	D, E, F, G
4	D	E, F, G
5	E	F, G
6	F	G
7	G	(empty)

Note: the queue at the start is [A]. In the table above, the "Queue" column shows the queue after the visited node has been removed and its children added.

2. Depth-First Search (DFS) Using the same graph,



Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

Answer (DFS from A):

DFS goes as deep as possible along one branch before backtracking. A stack (last in, first out) is used and the children are pushed so that the left child is explored first.

Iteration	Visited node (popped)	Stack after the iteration (top on the left)
1	A	B, C
2	B	D, E, C
3	D	E, C
4	E	C
5	C	F, G
6	F	G
7	G	(empty)

- **DFS traversal order:** A → B → D → E → C → F → G

D and E are leaves, so DFS backtracks to B and then continues with the unexplored branch C → F, G.

3. **Initial State :**

1 3 6
5 0 2
4 7 8

Goal State :

1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1	S0 1 3 6 5 0 2 4 7 8	S0	S1, S2, S3, S4
2	S1 1 0 6 5 3 2 4 7 8	S1, S2, S3, S4	S2, S3, S4, S5, S6
3	S2 1 3 6 5 7 2 4 0 8	S2, S3, S4, S5, S6	S3, S4, S5, S6, S7, S8
4	S3 1 3 6 0 5 2 4 7 8	S3, S4, S5, S6, S7, S8	S4, S5, S6, S7, S8, S9, S10
5	S4 1 3 6 5 2 0 4 7 8	S4, S5, S6, S7, S8, S9, S10	S5, S6, S7, S8, S9, S10, S11, S12

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

Answer (8-puzzle BFS):

Assumptions: a move slides a tile into the blank, which is the same as moving the blank 0 Up, Down, Left or Right, tried in that order. BFS keeps a visited list so a state that has already been generated is not added to the queue again. States are named S0, S1, S2 ... in the order they are generated (S0 = initial state). In the state diagrams "/" separates the rows of the 3 × 3 board.

Legend of the states used in the table:

State	Board (rows separated by /)	How it was generated
S0	1 3 6 / 5 0 2 / 4 7 8	Initial state
S1	1 0 6 / 5 3 2 / 4 7 8	S0, blank Up
S2	1 3 6 / 5 7 2 / 4 0 8	S0, blank Down
S3	1 3 6 / 0 5 2 / 4 7 8	S0, blank Left
S4	1 3 6 / 5 2 0 / 4 7 8	S0, blank Right
S5, S6	0 1 6 / 5 3 2 / 4 7 8 ; 1 6 0 / 5 3 2 / 4 7 8	S1, blank Left, Right

S7, S8	1 3 6 / 5 7 2 / 0 4 8 ; 1 3 6 / 5 7 2 / 4 8 0	S2, blank Left, Right
S9, S10	0 3 6 / 1 5 2 / 4 7 8 ; 1 3 6 / 4 5 2 / 0 7 8	S3, blank Up, Down
S11, S12	1 3 0 / 5 2 6 / 4 7 8 ; 1 3 6 / 5 2 8 / 4 7 0	S4, blank Up, Down

The table above shows the first five iterations. The goal is at depth 8 in the search tree, so BFS must finish all shallower levels first (depth 0 has 1 node, depth 1 has 4, depth 2 has 8, depth 3 has 8, depth 4 has 16, depth 5 has 32, depth 6 has 60, depth 7 has 72), and only then reaches the goal. Writing all the iterations is not practical, so the table is limited to the first five.

- **Which node is expanded in each iteration?** Level by level. Iteration 1 expands S0 (initial state), iterations 2–5 expand S1, S2, S3 and S4 (the four states at depth 1, in the order Up, Down, Left, Right). Iterations 6–13 expand the 8 states at depth 2, and so on, in first-in-first-out order. The goal state is reached (taken from the queue) in iteration 311.
- **How many states are generated in total?** With the usual BFS (goal test when a node is removed from the queue), 311 states are expanded and 500 states are generated in total (including the initial state). If the goal test is done as soon as a state is generated, the goal is found while expanding the 187th node, with 311 states generated by then.

Complete solution path (8 moves, depth 8):

Move	**Board after the move (rows separated by /)**
Start	1 3 6 / 5 0 2 / 4 7 8
Right	1 3 6 / 5 2 0 / 4 7 8
Up	1 3 0 / 5 2 6 / 4 7 8
Left	1 0 3 / 5 2 6 / 4 7 8
Down	1 2 3 / 5 0 6 / 4 7 8
Left	1 2 3 / 0 5 6 / 4 7 8
Down	1 2 3 / 4 5 6 / 0 7 8
Right	1 2 3 / 4 5 6 / 7 0 8
Right	1 2 3 / 4 5 6 / 7 8 0 (GOAL)

Moves of the blank: Right, Up, Left, Down, Left, Down, Right, Right. The path cost is 8.

Why does BFS always find the shortest solution? BFS expands every state at depth d before any state at depth d + 1. In the 8-puzzle each move has the same cost (1), so the cost of a path equals its number of moves, which is the same as its depth. The first time BFS reaches the goal is therefore at the smallest possible depth, and no shorter solution can exist, since a shorter one would have been found at a shallower level. The branching factor is finite (at most 4) and the state space is finite, so BFS is also complete. This is why BFS is optimal for unweighted problems like this one, though it needs a lot of memory because it stores every state of the frontier.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.